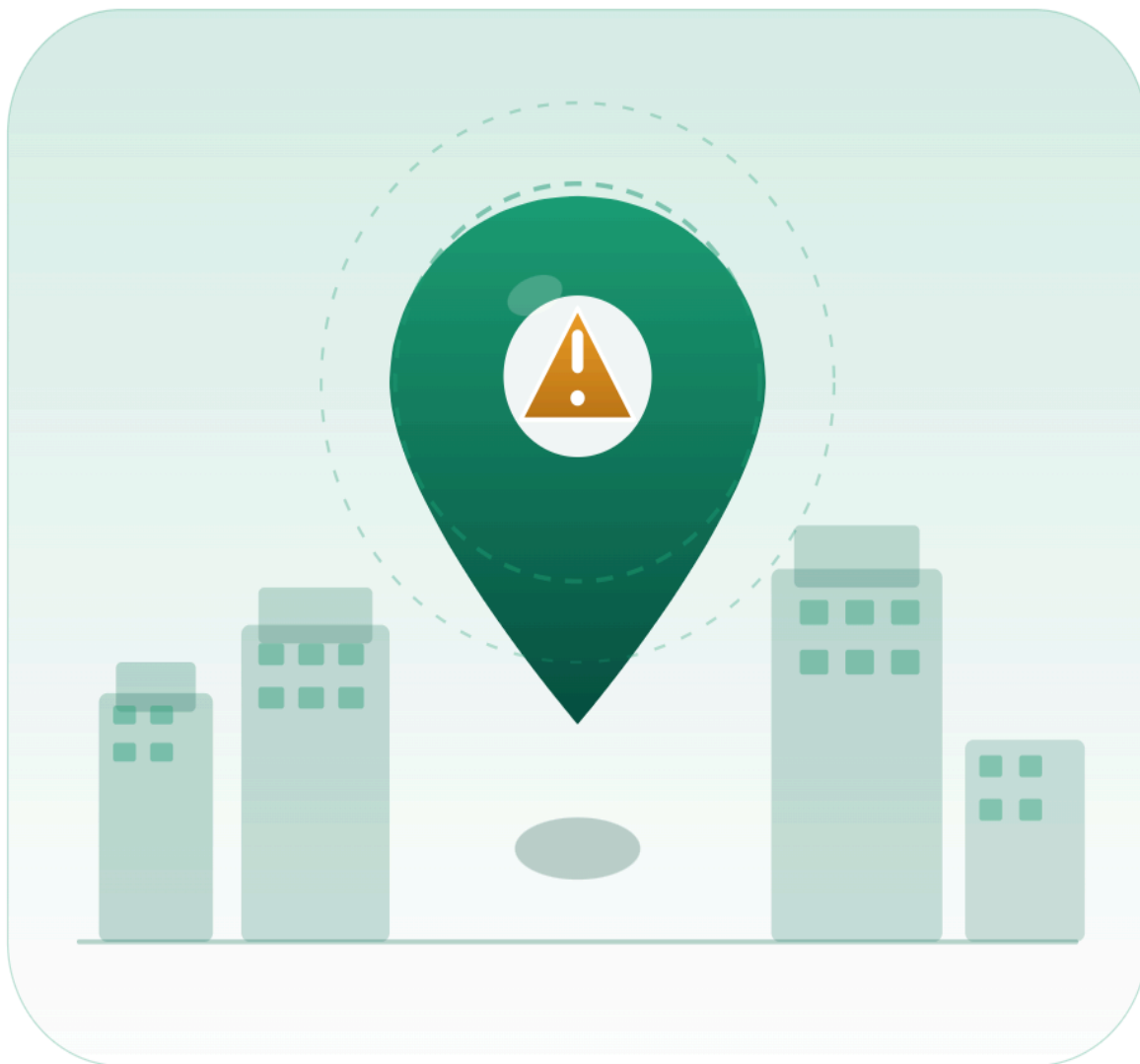


UrbiConnect

Documentación técnica

Aplicación de gestión de incidencias ciudadanas en el pueblo de Cantillana



Sergio Algarrada Miranda
17 de mayo de 2026
Versión 2.0

1. DESCRIPCIÓN COMPLETA DEL SISTEMA	3
1.1. Propósito y contexto	3
1.2. Funcionamiento general	3
2. ARQUITECTURA DEL SISTEMA	3
2.1. Explicación de la arquitectura	3
2.2. Diagrama UML de arquitectura	4
3. MODELO DE DATOS FINAL	4
3.1. Explicación del modelo	4
3.2. Diagrama UML del modelo de datos	5
3.3. Rutas de base de datos (Sustituto de API REST)	5
4. MANUAL TÉCNICO AVANZADO	6
4.1. Entorno de desarrollo y stack tecnológico	6
4.2. Lógica de servicios (Desglose de funcionalidades)	6
4.3. Gestión de la geolocalización	7
4.4. Optimización multimedia y almacenamiento	7
4.5. Seguridad en el lado del servidor (Firestore Rules)	7
4.6. Despliegue y mantenimiento PWA	7
5. PRUEBAS FUNCIONALES REALIZADAS	8

1. DESCRIPCIÓN COMPLETA DEL SISTEMA

1.1. Propósito y contexto

UrbiConnect es una solución tecnológica integral diseñada para transformar la comunicación entre la ciudadanía de Cantillana y su ayuntamiento. El sistema surge de la observación de una problemática común en la gestión municipal: la lentitud en la detección y resolución de desperfectos en la vía pública (baches, mobiliario roto, incidencias de limpieza o alumbrado, etc). La aplicación permite que cualquier vecino, utilizando su dispositivo smartphone o un navegador web, pueda generar un reporte detallado que llega de forma inmediata al departamento correspondiente. Al digitalizar este proceso, eliminamos la burocracia telefónica y presencial, permitiendo una gestión mucho más dinámica y transparente.

1.2. Funcionamiento general

El sistema se basa en un flujo de trabajo colaborativo:

- **Reporte:** El ciudadano crea una incidencia aportando hasta el último detalle (fotos, descripción y geoposicionamiento automático).
- **Asignación:** El sistema, mediante un panel de administración, clasifica la incidencia. Los operarios municipales (responsables) reciben solo aquellas que competen a su área técnica.
- **Seguimiento:** Se establece un canal de comunicación bidireccional (chat) para resolver dudas sobre la incidencia.
- **Resolución:** Una vez reparada, el responsable actualiza el estado y el ciudadano recibe una notificación automática, informando de que su aviso ha sido atendido con éxito.

2. ARQUITECTURA DEL SISTEMA

2.1. Explicación de la arquitectura

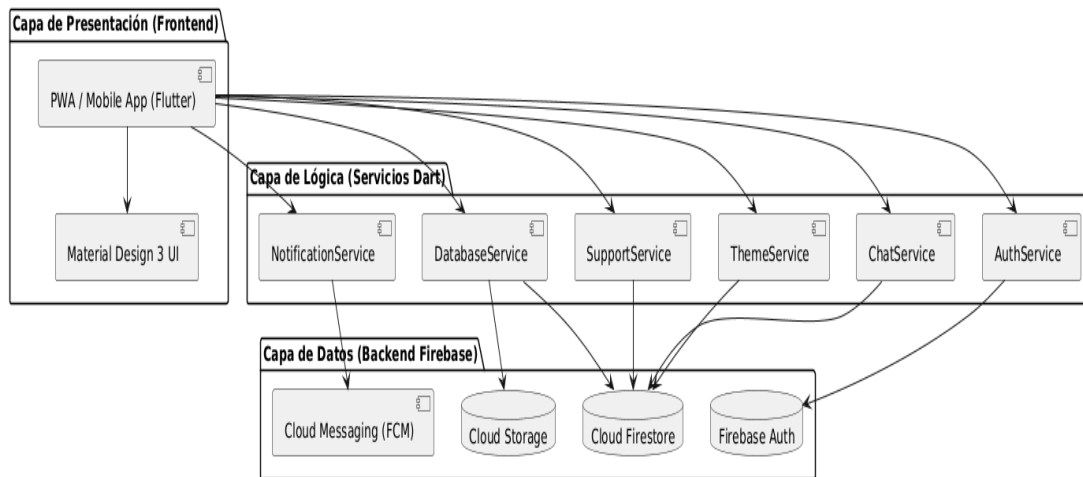
Se ha seleccionado una arquitectura Serverless (BaaS - Backend as a Service) utilizando el ecosistema de Firebase e integrada con el framework Flutter. Esta elección permite que la lógica de negocio resida en una única base de código que se compila de forma nativa para múltiples plataformas (PWA Web y Android).

La arquitectura se divide en tres capas fundamentales:

- **Capa de presentación:** Desarrollada con Flutter y siguiendo las guías de diseño de Material Design 3, garantizando una interfaz adaptativa (responsive) y accesible.
- **Capa de lógica (servicios):** Implementada en Dart, donde cada servicio encapsula una funcionalidad específica, facilitando el mantenimiento y las pruebas unitarias.

- **Capa de datos:** Alojada en Google Cloud, que gestiona la persistencia en tiempo real, el almacenamiento de archivos binarios y la autenticación federada.

2.2. Diagrama UML de arquitectura



3. MODELO DE DATOS FINAL

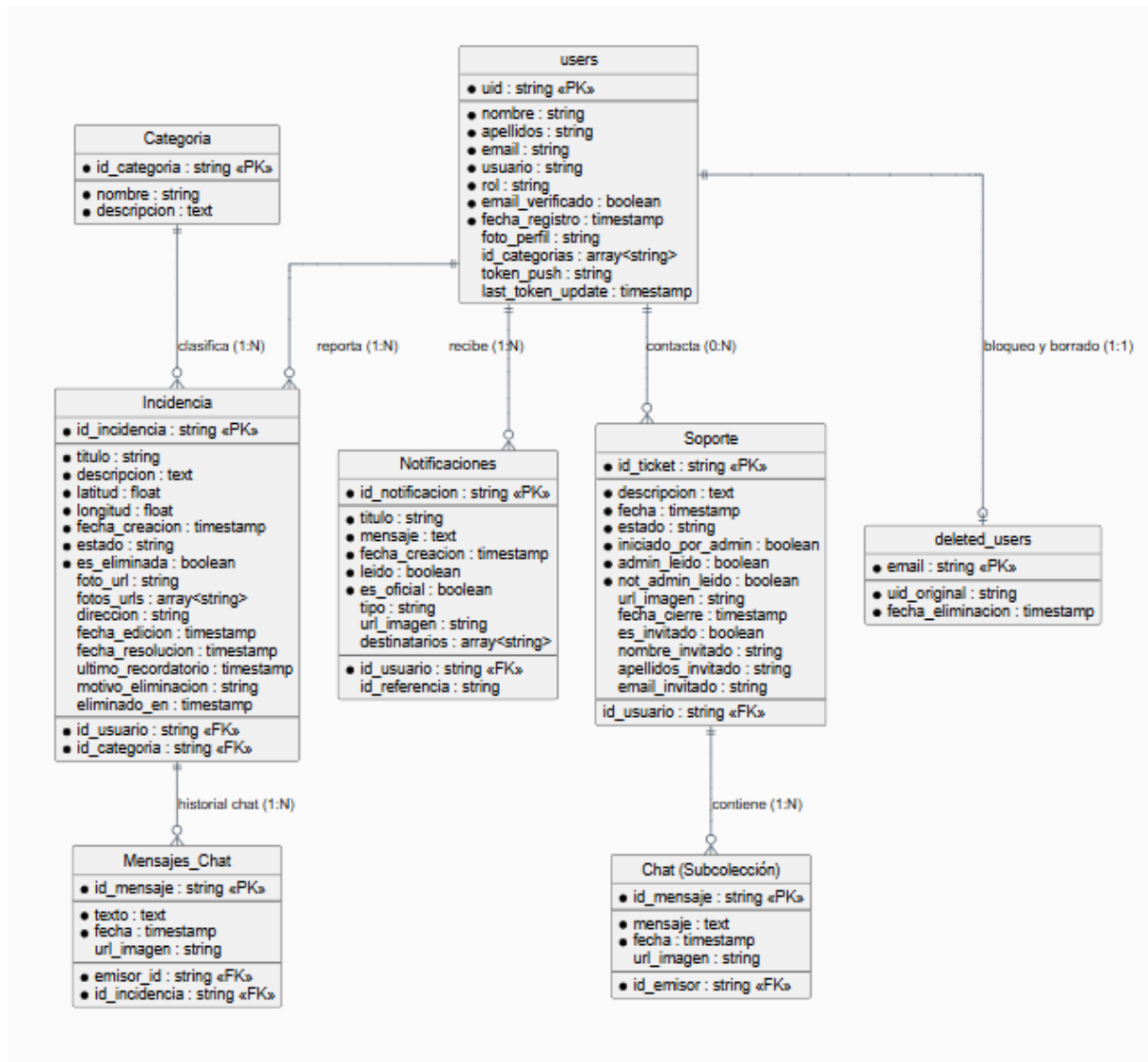
3.1. Explicación del modelo

UrbiConnect emplea Cloud Firestore, una base de datos NoSQL basada en documentos. La información no se almacena en tablas rígidas, sino en colecciones de documentos JSON, lo que permite que el sistema sea extremadamente rápido y flexible ante cambios futuros. Las diferentes colecciones y subcolecciones de datos de la aplicación son:

- **users:** Almacena perfiles de usuario, roles, asignación de áreas (para responsables municipales) y tokens de notificaciones push.
- **deleted_users:** Registro de correos electrónicos bloqueados de cuentas eliminadas, diseñado para impedir que infractores o usuarios borrados vuelvan a registrarse en el sistema.
- **Incidencia:** El núcleo del sistema con todos los datos técnicos del reporte (geolocalización, estado, historial de fechas, arrays de múltiples fotografías y parámetros de borrado lógico o soft delete).
- **Categoria:** Diccionario de áreas municipales (Limpieza, Tráfico, etc.) utilizado para clasificar y derivar los reportes.
- **Mensajes_Chat:** Historial de conversación asíncrona asociado directamente a la gestión y resolución de una incidencia.
- **Notificaciones:** Registro de alertas enviadas al buzón del usuario (alertas automáticas de estado, recordatorios y comunicados oficiales masivos o broadcast).

- **Soporte:** Gestión de consultas técnicas y tickets de asistencia dirigidos a los administradores (permite tanto usuarios registrados como invitados).
- **Chat** (Subcolección de Soporte): Historial interno de los mensajes y fotografías intercambiados entre el administrador y el usuario dentro de un ticket de soporte específico.

3.2. Diagrama UML del modelo de datos



3.3. Rutas de base de datos (Sustituto de API REST)

Dado que el sistema utiliza una arquitectura Serverless (BaaS) con Firebase en lugar de una API REST tradicional, las peticiones de red (endpoints) se sustituyen por operaciones directas sobre las rutas (paths) de los documentos en Firestore. Las rutas principales de acceso a los datos son:

- /users/{userId}
- /deleted_users/{email}
- /Categoria/{categoriaId}
- /Incidencia/{incidenciaId}
- /Mensajes_Chat/{mensajeId}
- /Notificaciones/{notificacionId}
- /Soporte/{ticketId}
- /Soporte/{ticketId}/Chat/{mensajeId}

4. MANUAL TÉCNICO AVANZADO

4.1. Entorno de desarrollo y stack tecnológico

El proyecto se ha construido bajo el entorno de Flutter 3.35.4. Es indispensable contar con las siguientes herramientas para el mantenimiento del código:

- **Dart SDK**: Corazón del lenguaje de programación. Estamos usando la versión 3.9.2.
- **Firebase Core**: Para la vinculación del proyecto con la nube de Google.
- **Google Maps SDK**: Para el renderizado de mapas y captura de coordenadas satelitales.
- **Repositorio de código**: Todo el código fuente de la aplicación está organizado de forma modular y versionado mediante Git. Puedes consultar el repositorio oficial del proyecto en el siguiente enlace: <https://github.com/sergioam26/Urbiconnect.git>.

4.2. Lógica de servicios (Desglose de funcionalidades)

El sistema aplica el principio de responsabilidad única mediante servicios especializados:

- **AuthService**: Implementa el flujo de autenticación "OAuth2" con Google y el sistema nativo. Gestiona la persistencia de la sesión y el borrado de tokens al cerrar sesión.
- **DatabaseService**: Utiliza Streams de Firestore para permitir la reactividad. Cuando un responsable cambia el estado de una incidencia, el ciudadano ve el cambio en su pantalla sin necesidad de refrescar la aplicación.
- **ChatService**: Permite la comunicación en tiempo real. Los mensajes se guardan asociados al ID de la incidencia y permiten envío de imágenes para mayor detalle técnico.
- **NotificationService**: Gestiona tanto notificaciones en la nube como el "Broadcast" municipal, enviando avisos masivos a grupos específicos de destinatarios.
- **SupportService**: Implementa un sistema de tickets para usuarios invitados (no registrados), permitiendo que cualquier persona reporte fallos de la propia app.

- **ThemeService**: Implementa el estándar de Material You, ajustándose a las preferencias del sistema o del usuario (Modo oscuro/claro).

4.3. Gestión de la geolocalización

La precisión es crítica. El sistema utiliza el paquete geolocator con una configuración de LocationAccuracy.high.

El flujo es el siguiente: La app solicita permiso de ubicación al abrir el formulario de incidencia. Si el GPS está desactivado, el sistema intenta triangular mediante torres de red móvil para asegurar que el técnico municipal siempre reciba una ubicación aproximada.

4.4. Optimización multimedia y almacenamiento

Para evitar gastos excesivos en la nube y mejorar la experiencia de usuario con poca cobertura se toman las siguientes medidas:

- **Compresión**: Antes de subir cualquier imagen a Firebase Storage, se realiza una compresión en el cliente reduciendo la calidad al 80% y limitando el ancho a 1024px.
- **Multi-foto**: El campo fotos_urls permite almacenar un historial visual de la incidencia (ej. foto antes de la reparación y foto después).

4.5. Seguridad en el lado del servidor (Firestore Rules)

La seguridad reside en las Security Rules de Firebase, no en el cliente. Cualquier ciudadano autenticado puede leer la colección de Categorías para poder crear incidencias, pero solo el administrador puede crear, modificar o borrar esas categorías. Además, un usuario solo puede acceder o modificar sus propios datos. Colecciones sensibles como el historial de Mensajes_Chat o las Notificaciones exigen que el usuario esté debidamente autenticado y validan que el campo id_usuario o emisor_id coincida exactamente con su UID. Ningún ciudadano puede alterar su propio campo rol para escalar privilegios de forma maliciosa.

4.6. Despliegue y mantenimiento PWA

La aplicación se compila para Web mediante el comando **flutter build web --web-renderer canvaskit**. Se despliega sobre un servidor seguro HTTPS. Esto es obligatorio: sin un certificado SSL válido, los navegadores modernos por seguridad bloquean el acceso al sensor GPS y a la cámara, componentes vitales para UrbiConnect.

5. PRUEBAS FUNCIONALES REALIZADAS

Para validar el sistema se han realizado pruebas de caja negra:

- **Control de Login:** Se comprobó que si un usuario introduce un correo electrónico mal formado o una contraseña demasiado corta, el sistema responde correctamente mostrando un aviso (SnackBar) con el error de validación.
- **Campos obligatorios:** Se validó que al intentar enviar una incidencia dejando el título vacío, el sistema lo impide marcando el campo en rojo o deshabilitando el botón de envío.
- **Previsualización de fotografías:** Se verificó que, al seleccionar una foto desde la galería del dispositivo, la imagen aparezca correctamente en un widget en forma de miniatura antes de ser subida a la base de datos.
- **Diferenciación de roles:** Se probó el inicio de sesión alternando entre una cuenta de Ciudadano y una de Administrador, confirmando que la interfaz se adapta al nivel de permisos (el Ciudadano visualiza el botón "+" para reportar, mientras que el Administrador accede al botón de "Gestionar usuarios").
- **Filtros de lista por áreas:** Se validó que un Responsable Municipal únicamente tenga acceso a las incidencias de sus áreas de competencia, de forma que un responsable de "Limpieza" solo ve esos reportes y no los de otras áreas como alumbrado.
- **Fluidez y sincronización del chat:** Se constató que al enviar un mensaje de chat desde un dispositivo, este aparece de manera instantánea en el otro perfil con el nombre de emisor correcto, confirmando el funcionamiento en tiempo real.
- **Modo Invitado en Soporte:** Se comprobó que es posible acceder al apartado de Soporte técnico con la sesión cerrada, permitiendo rellenar y enviar el formulario de contacto sin necesidad de tener una cuenta activa.
- **Validación de nombre de usuario único:** Se probó que al intentar registrar una cuenta (o editar un perfil) con un nombre de usuario que ya pertenece a otra persona, el sistema lo detecta en tiempo real y muestra el aviso "El nombre de usuario ya está en uso".
- **Bloqueo temporal por falta de verificación:** Se comprobó que un usuario recién registrado mediante correo y contraseña no puede acceder a las funcionalidades de la aplicación hasta que pulsa el enlace de su correo, mostrándose una pantalla de bloqueo con el mensaje "Verifica tu correo".
- **Restricción de extensiones en subida de archivos:** Se verificó que al intentar adjuntar un archivo que no sea una imagen válida (como un PDF o un documento) en un comunicado o incidencia, el sistema lo bloquea y alerta al usuario indicando "no es una imagen permitida".
- **Límite de spam en recordatorios:** Se validó que, para no colapsar al ayuntamiento, un ciudadano solo puede enviar un recordatorio de una incidencia si han pasado al menos 3 días desde el último. Si lo intenta antes, recibe el aviso "Solo puedes enviar un recordatorio cada 3 días".

- ***Inmutabilidad de incidencias resueltas o eliminadas***: Se probó que cuando una incidencia cambia a estado "Resuelta" o "Eliminada", los botones de edición y borrado desaparecen para el ciudadano, mostrando una advertencia de que la incidencia ya no puede ser modificada.
- ***Seguridad en el cambio de correo electrónico (reautenticación)***: Se verificó que si un usuario intenta cambiar su dirección de email desde la pantalla de edición de perfil, el sistema levanta un cuadro de diálogo exigiéndole que introduzca su contraseña actual por motivos de seguridad.
- ***Control de permisos y fallos del GPS***: Se comprobó el comportamiento de la app cuando el usuario deniega los permisos de ubicación o tiene el GPS desactivado. El sistema no se "rompe", sino que muestra un aviso de denegación y permite buscar la dirección manualmente mediante la API de Nominatim/OpenStreetMap.
- ***Comprobación del sistema de baneo (deleted_users)***: Se testeó que si el administrador elimina la cuenta de un usuario (guardando su email en deleted_users), esa persona no puede volver a registrarse ni iniciar sesión con ese mismo correo, recibiendo el mensaje de cuenta bloqueada.
- ***Apertura automática de tickets de soporte***: Se verificó que si un ciudadano tiene un ticket de soporte cerrado y el Administrador decide responderle, el estado del ticket cambia automáticamente de "Cerrado" a "Abierto" de nuevo.
- ***Validación de contraseñas cruzadas en registro***: Se probó el formulario de registro introduciendo dos contraseñas distintas, comprobando que el sistema no permite avanzar y marca el error "Las contraseñas no coinciden", además de exigir un mínimo de 6 caracteres.